# UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

## FACULTAD DE CONTADURÍA Y ADMINISTRACIÓN

SISTEMA DE UNIVERSIDAD ABIERTA

**Alumno: Emanuel Marzano Villalobos**

**Programación De Dispositivos Móviles**

Cristian Cardoso Arellano

Grupo 9697

# m5_02

29/10/2026

Enlace a repositorio de la actividad: https://github.com/Mr-X22/android-2027-1/tree/main/M5_02

# 1. Introducción

Muchos problemas de programación consisten en leer una gran cantidad de datos, procesarlos y obtener un resultado. Esta actividad presenta uno de ellos en forma de historia: los elfos de Papá Noel anotan las calorías de los alimentos que lleva cada uno, y es necesario encontrar al elfo que lleva más calorías.

Detrás de la historia, el problema es sencillo: el archivo de entrada contiene números organizados en grupos, separados por líneas en blanco. Cada grupo corresponde a un elfo. Hay que sumar los números de cada grupo y encontrar la suma más grande.

Para resolverlo se desarrolló una aplicación Android en lenguaje Java que lee el archivo de datos proporcionado por el asesor, lo recorre línea por línea con un ciclo while, utiliza sentencias condicionales if/else para decidir qué hacer con cada línea e imprime el resultado en Logcat.

# 2. Objetivo

Resolver el problema del conteo de calorías en lenguaje Java dentro de Android Studio, aplicando estructuras de control de flujo (ciclo while y sentencias if/else) para leer y procesar el archivo de entrada, y responder la pregunta: ¿cuántas calorías totales lleva el elfo que lleva más calorías?

# 3. Análisis del problema

## 3.1. ¿Qué pide el problema?

Con el ejemplo del enunciado se entiende mejor:

```
1000  ┐
2000  ├ elfo 1 = 6000
3000  ┘

4000  → elfo 2 = 4000

5000  ┐
6000  ┘ elfo 3 = 11000

7000  ┐
8000  ├ elfo 4 = 24000   ← el mayor
9000  ┘

10000 → elfo 5 = 10000
```

La respuesta del ejemplo es 24000, que lleva el cuarto elfo. Lo mismo hay que hacer con el archivo real, que tiene más de 2,200 líneas.

## 3.2. Algoritmo de la solución

Antes de programar, se definió el algoritmo en pasos:

- **1.** Iniciar cuatro variables en cero: la suma del elfo actual, la suma más grande, el número del elfo actual (en 1) y el número del elfo con más calorías.
- **2.** Leer el archivo una línea a la vez, mientras queden líneas.
- **3.** Si la línea tiene un número, sumarlo al elfo actual.
- **4.** Si la línea está vacía, el elfo terminó: si su suma es mayor que la más grande, guardarla como la nueva más grande junto con el número del elfo. Después pasar al siguiente elfo y reiniciar la suma en cero.
- **5.** Al terminar el archivo, revisar también al último elfo, porque después de él ya no hay una línea vacía.
- **6.** Mostrar el resultado.

**Una comparación para entenderlo**

Es como revisar las mochilas de varios excursionistas para saber quién lleva más comida. Vas sacando los alimentos de una mochila y sumando sus calorías; cuando terminas con esa mochila, comparas el total con el récord que llevas anotado y, si lo supera, anotas el nuevo récord. Luego pasas a la siguiente mochila empezando la suma desde cero.

# 4. Desarrollo

## 4.1. Creación del proyecto y carpeta assets

Se creó el proyecto M5-02 con la plantilla Empty Views Activity y el lenguaje Java. Para que la aplicación pueda leer el archivo de datos, se creó la carpeta assets mediante clic derecho en app → New → Folder → Assets Folder. Como se vio en la actividad M1-01, los archivos de esta carpeta conservan su nombre y contenido original, y la aplicación puede abrirlos tal cual.

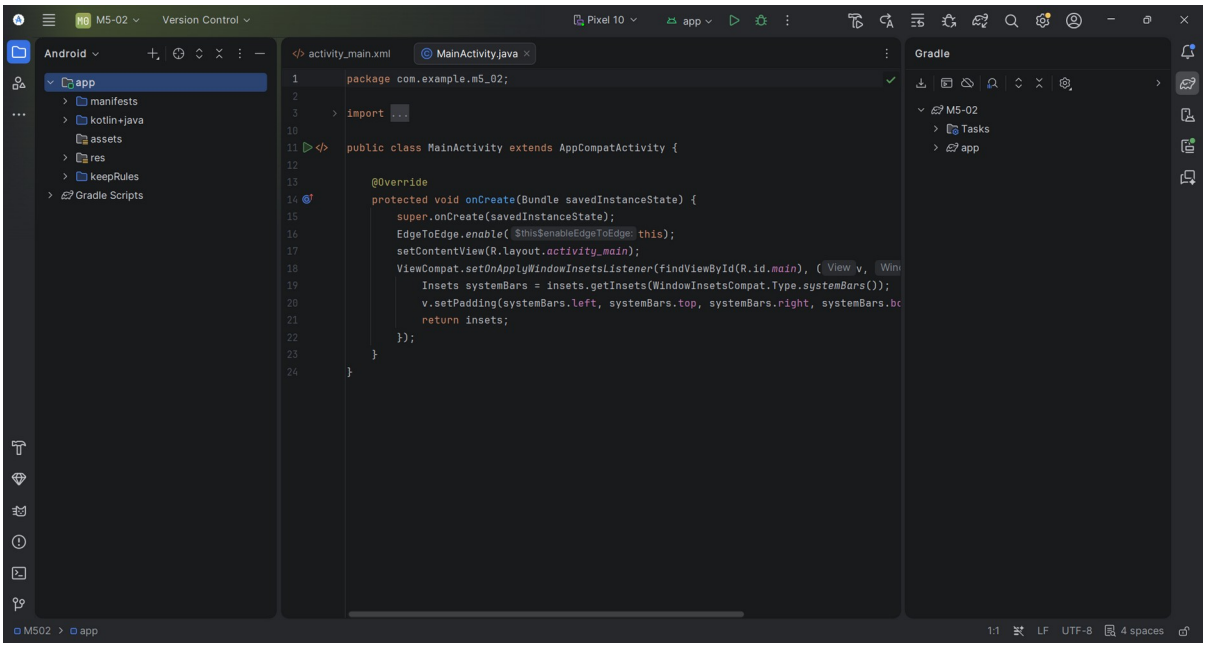



*Figura 1. Carpeta assets creada en el proyecto M5-02.*

Después se descargó el archivo de entrada desde el repositorio del asesor (inputs/input-M5-02), se renombró como input.txt y se colocó dentro de la carpeta assets.

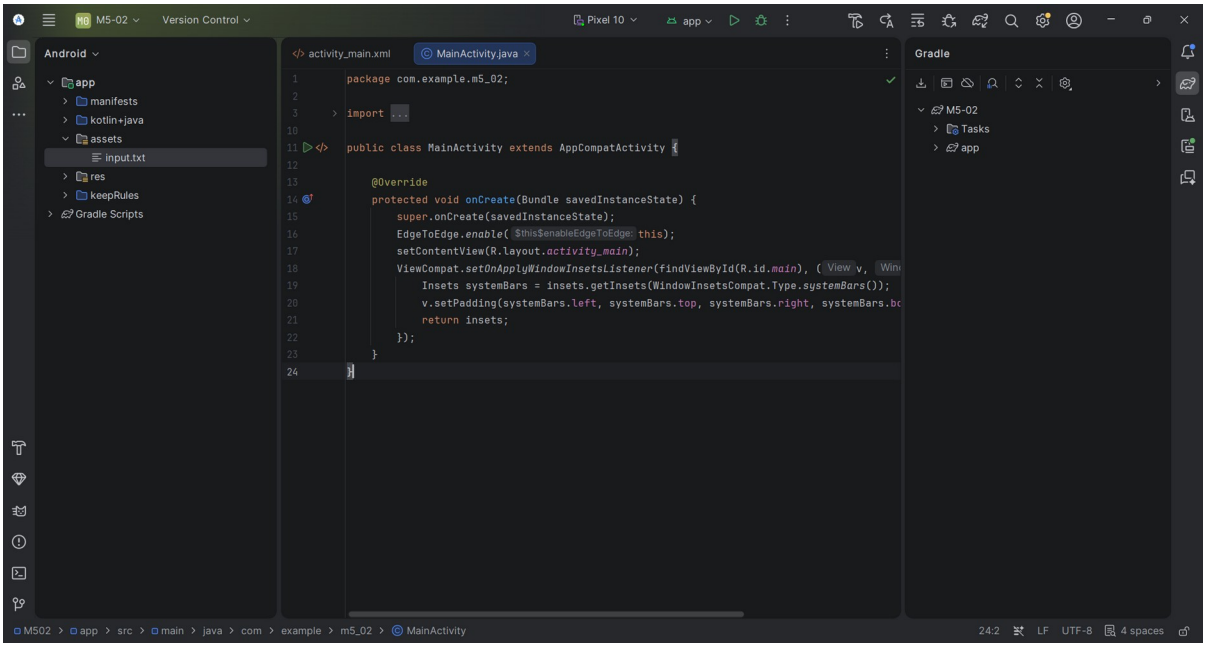



*Figura 2. Archivo input.txt dentro de la carpeta assets.*

## 4.2. Código de la solución

### *Constantes y llamada desde onCreate()*

```
private static final String TAG = "M5_02";
private static final String ARCHIVO = "input.txt";

// dentro de onCreate(), al final:
resolverCalorias();
```

- **`TAG`**: etiqueta para filtrar los mensajes en Logcat.
- **`ARCHIVO`**: nombre del archivo de datos. Al estar en una constante, si el archivo cambiara de nombre bastaría con modificar esta línea.
- **`resolverCalorias()`**: la solución se escribió en un método aparte para mantener onCreate() ordenado. Se llama al iniciar la aplicación.

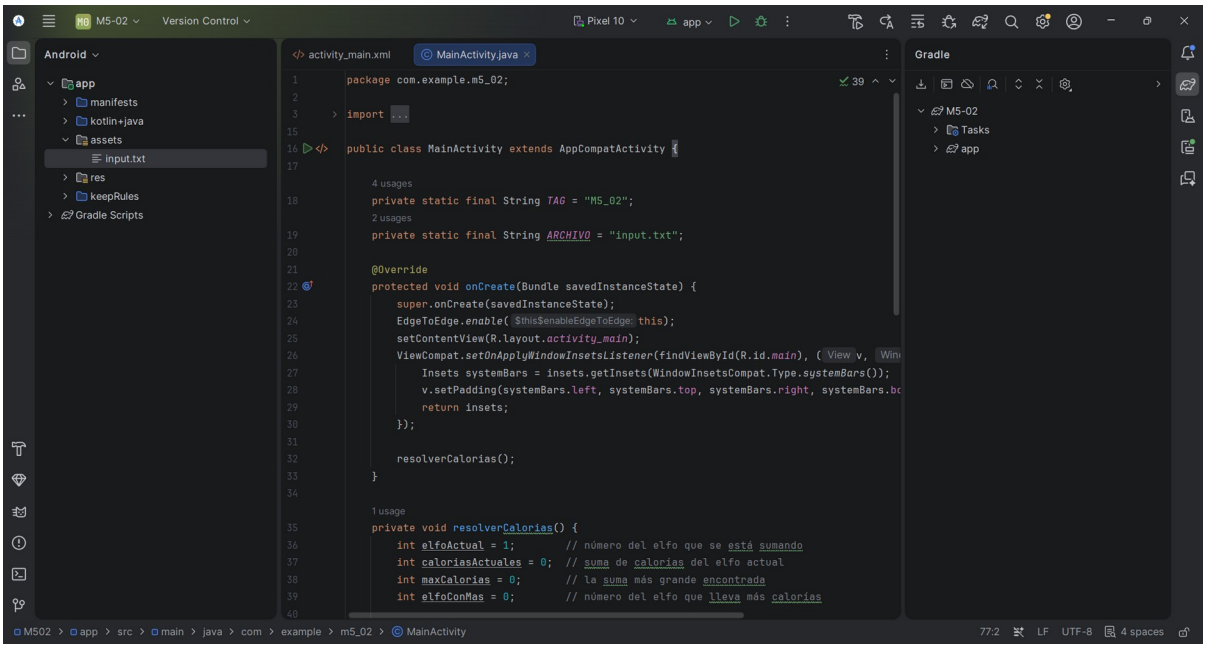



*Figura 3. Constantes, onCreate() y variables del método resolverCalorias().*

### *Variables del método*

```
int elfoActual = 1;
int caloriasActuales = 0;
int maxCalorias = 0;
int elfoConMas = 0;
```

- **`elfoActual`**: número del elfo que se está sumando. Empieza en 1 y aumenta en cada línea vacía.
- **`caloriasActuales`**: suma de las calorías del elfo actual. Vuelve a 0 cada vez que empieza un elfo nuevo.
- **`maxCalorias`**: la suma más grande encontrada hasta el momento; funciona como un "récord".

- **`elfoConMas`**: número del elfo que tiene el récord.

***Lectura del archivo línea por línea***

```
try (BufferedReader lector = new BufferedReader(
        new InputStreamReader(getAssets().open(ARCHIVO)))) {

    String linea;
    while ((linea = lector.readLine()) != null) {
        linea = linea.trim();
        ...
    }
    ...
} catch (IOException e) {
    Log.e(TAG, "No se pudo leer el archivo " + ARCHIVO, e);
}
```

- **`getAssets().open(ARCHIVO)`**: abre el archivo input.txt desde la carpeta assets.
- **`InputStreamReader`**: convierte los datos del archivo (bytes) en caracteres de texto.
- **`BufferedReader`**: es un “lector” que permite leer el texto línea por línea con el método readLine().
- **`while ((linea = lector.readLine()) != null)`**: es un ciclo while. En cada vuelta lee la siguiente línea y la guarda en la variable linea. Cuando el archivo se acaba, readLine() regresa null (“nada”) y el ciclo termina. Se usa while en lugar de for porque no se sabe de antemano cuántas líneas tiene el archivo.
- **`linea.trim()`**: elimina los espacios invisibles que pudiera haber al inicio o al final de la línea, para que una línea con solo espacios cuente como vacía.
- **`try (…) { … } catch (IOException e)`**: si ocurre un problema al leer (por ejemplo, que el archivo no exista), en lugar de que la aplicación se detenga, se ejecuta el bloque catch, que imprime un mensaje de error en Logcat. El paréntesis después de try cierra el archivo automáticamente al terminar.

***Decisión con if / else***

```
if (linea.isEmpty()) {
    // Línea en blanco: terminó el inventario de un elfo
    if (caloriasActuales > maxCalorias) {
        maxCalorias = caloriasActuales;
        elfoConMas = elfoActual;
    }
    elfoActual++;
    caloriasActuales = 0;
} else {
    // Línea con número: se suma al elfo actual
    caloriasActuales += Integer.parseInt(linea);
}
```

- **`if (linea.isEmpty())`**: revisa si la línea está vacía, lo que significa que terminó el inventario de un elfo.

- **`if (caloriasActuales > maxCalorias)`**: condición anidada: compara la suma del elfo que acaba de terminar con el récord. Si es mayor, se guarda como nuevo récord junto con el número del elfo.
- **`elfoActual++`**: pasa al siguiente elfo.
- **`caloriasActuales = 0`**: reinicia la suma para empezar a contar las calorías del nuevo elfo.
- **`else`**: si la línea no está vacía, contiene un número.
- **`Integer.parseInt(linea)`**: convierte el texto de la línea (por ejemplo, "1000") en un número entero que se puede sumar.
- **`+=`**: suma el número a lo que ya tenía caloriasActuales. Es lo mismo que escribir caloriasActuales = caloriasActuales + número.

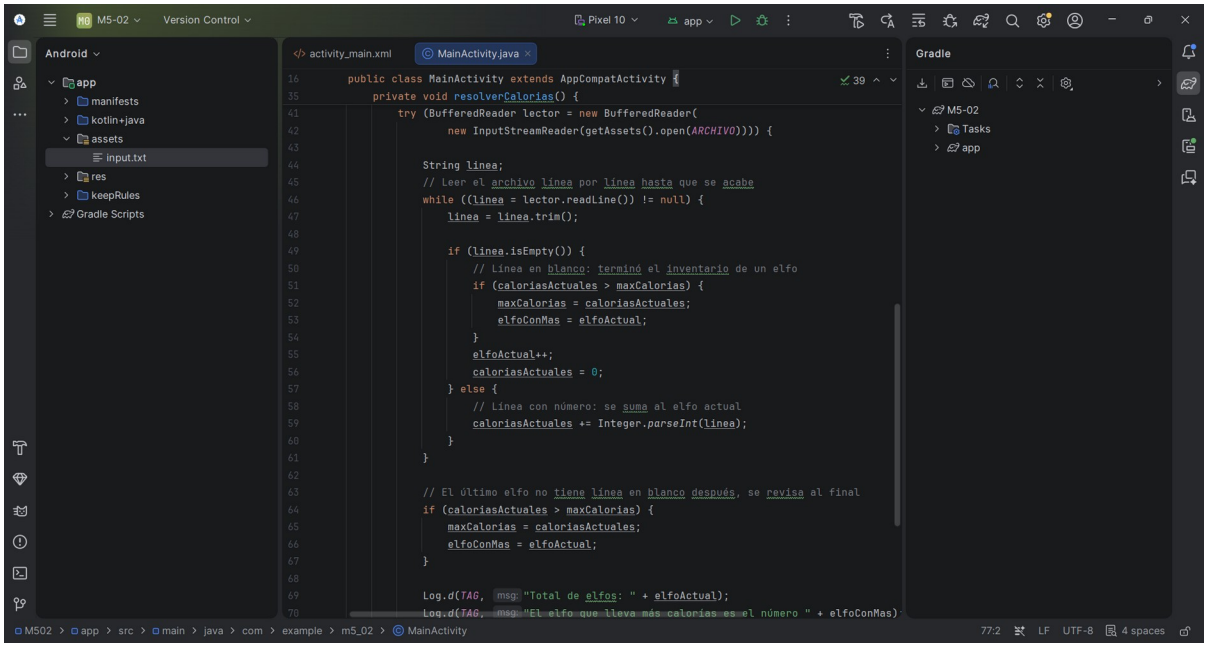



*Figura 4. Ciclo while con las sentencias if/else.*

### *Revisión del último elfo e impresión del resultado*

```
if (caloriasActuales > maxCalorias) {
    maxCalorias = caloriasActuales;
    elfoConMas = elfoActual;
}

Log.d(TAG, "Total de elfos: " + elfoActual);
Log.d(TAG, "El elfo que lleva más calorías es el número " + elfoConMas);
Log.d(TAG, "Calorías totales que lleva ese elfo: " + maxCalorias);
```

- **`Revisión final`**: el último elfo del archivo no tiene una línea vacía después, por lo que el ciclo termina sin haberlo comparado. Esta revisión, fuera del ciclo, evita que se quede sin contar. Sin ella, si el último elfo fuera el de más calorías, el resultado sería incorrecto.
- **`Log.d(…)`**: imprime en Logcat el total de elfos, el número del elfo ganador y sus calorías.

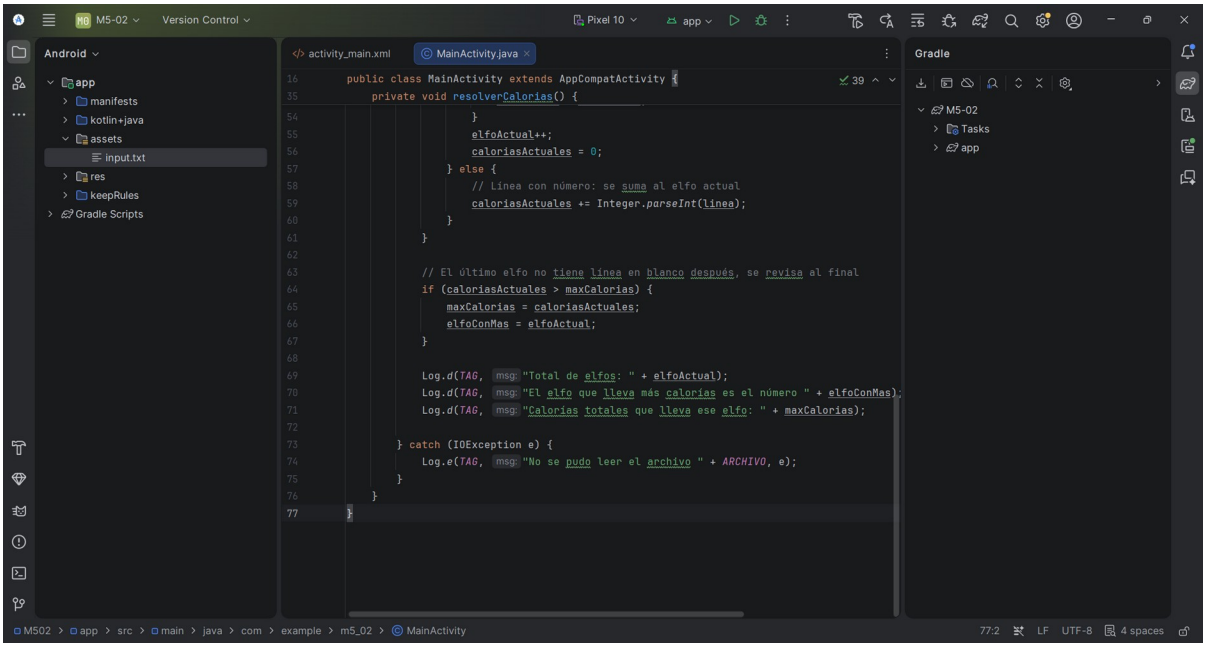



*Figura 5. Revisión del último elfo, impresión del resultado y bloque catch.*

### 4.3. Prueba de escritorio con el ejemplo

Para comprobar que la lógica es correcta, se siguió el algoritmo “a mano” con el ejemplo del enunciado. La tabla muestra cómo cambian las variables en cada línea:

| Línea | Contenido | caloriasActuales | maxCalorias | Qué ocurre |
|---|---|---|---|---|
| 1 | 1000 | 1000 | 0 | Se suma al elfo 1 |
| 2 | 2000 | 3000 | 0 | Se suma al elfo 1 |
| 3 | 3000 | 6000 | 0 | Se suma al elfo 1 |
| 4 | (vacía) | 0 | 6000 | Termina el elfo 1: 6000 es el mayor |
| 5 | 4000 | 4000 | 6000 | Se suma al elfo 2 |
| 6 | (vacía) | 0 | 6000 | Termina el elfo 2: 4000 no supera 6000 |
| 7–8 | 5000, 6000 | 11000 | 6000 | Se suman al elfo 3 |
| 9 | (vacía) | 0 | 11000 | Termina el elfo 3: nuevo mayor |
| 10–12 | 7000, 8000, 9000 | 24000 | 11000 | Se suman al elfo 4 |
| 13 | (vacía) | 0 | 24000 | Termina el elfo 4: nuevo mayor |
| 14 | 10000 | 10000 | 24000 | Se suma al elfo 5 |
| Fin | — | 10000 | 24000 | Revisión final: 10000 no supera 24000 |

El resultado es 24000, igual al que indica el enunciado, lo que confirma que el algoritmo funciona, incluida la revisión del último elfo.

### 4.4. Ejecución y resultado

La aplicación se ejecutó en el emulador Pixel 10 (Android 17, API 37). Como el resultado se imprime en Logcat, la pantalla de la aplicación conserva el texto de la plantilla. En Logcat se aplicó el filtro tag:M5_02.

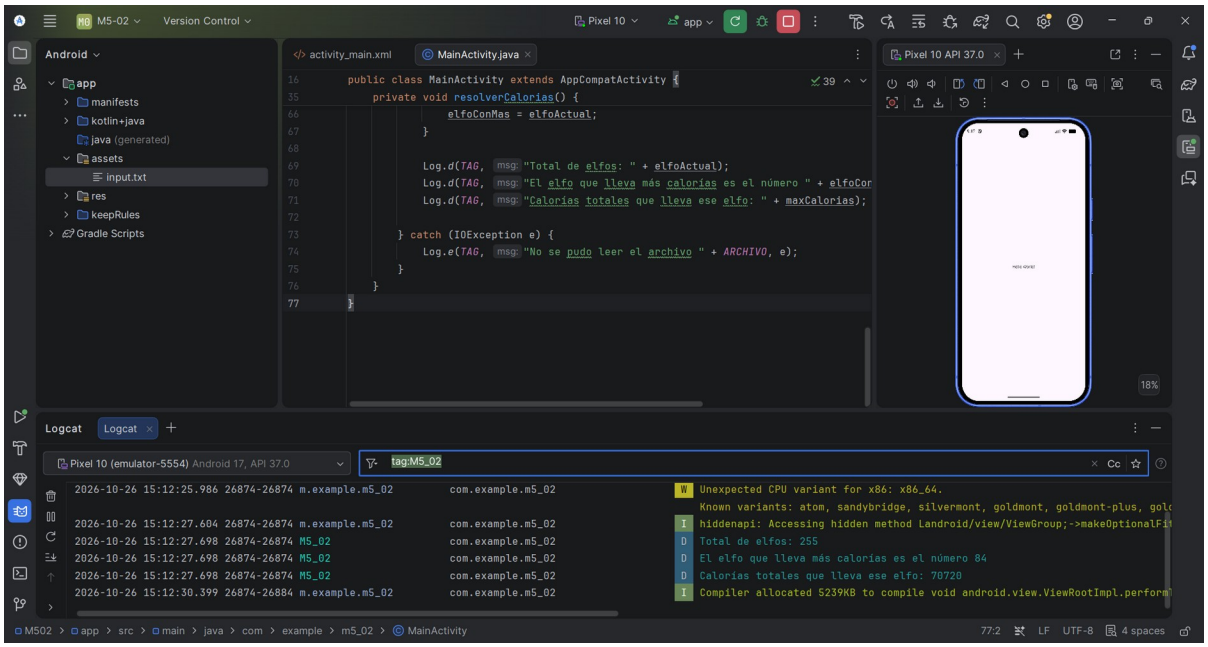



*Figura 6. Resultado de la ejecución en Logcat.*

| Dato | Resultado |
|---|---|
| Total de elfos en el archivo | 255 |
| Elfo que lleva más calorías | Elfo número 84 |
| Calorías totales de ese elfo | 70,720 |

**Respuesta a la pregunta:** el elfo que lleva más calorías es el número 84, y lleva un total de **70,720 calorías**.

Como verificación adicional, el archivo se procesó también por separado fuera de Android Studio, y se obtuvieron los mismos valores: 255 elfos y un máximo de 70,720 calorías en el elfo 84.

## 5. Conclusión

Aunque el enunciado parecía complicado por la historia de los elfos, al analizarlo resultó ser un problema de sumar grupos de números y encontrar el mayor. Escribir primero el algoritmo en pasos

facilitó mucho la programación, porque cada paso se convirtió directamente en una parte del código.

La actividad permitió aplicar el ciclo while, útil cuando no se sabe cuántas veces se va a repetir algo (en este caso, cuántas líneas tiene el archivo), y las sentencias if/else para decidir qué hacer con cada línea. También fue importante notar un detalle fácil de olvidar: el último elfo no tiene una línea vacía después, por lo que hubo que revisarlo por separado al terminar el ciclo.

Además, se aprendió a leer un archivo desde la carpeta assets con BufferedReader y a manejar posibles errores con try/catch, lo que hace que la aplicación no se detenga si el archivo no existe. La prueba de escritorio con el ejemplo del enunciado ayudó a confirmar que la lógica era correcta antes de procesar el archivo real.

## 6. Referencias


Alex Lee. (2019, 8 de agosto). *parseInt Java tutorial - String to Integer #56* [Video]. YouTube. https://www.youtube.com/watch?v=Pnaqn6GOyzU

Cardoso, C. (2023). *input-M5-02* [Archivo de datos]. GitHub. https://github.com/CristianCardosoA/FCA-iOS-2023-1/blob/main/inputs/input-M5-02

Charly Cimino. (2023, 16 de marzo). *PRUEBAS de ESCRITORIO. Clave para entender algoritmos* [Video]. YouTube. https://www.youtube.com/watch?v=6gAwS0FgzUY

*Cómo hacer una prueba de escritorio*. (2014, 11 de agosto). Pensamientos de Aztlek. https://aztlek.org/2014/08/11/como-hacer-una-prueba-de-escritorio/

Google. (s. f.-a). *AssetManager*. Android Developers. https://developer.android.com/reference/android/content/res/AssetManager

Google. (s. f.-b). *Integer*. Android Developers. https://developer.android.com/reference/java/lang/Integer

Miranda, M. (2020, 24 de febrero). *Android / Acceso a archivos, recursos*. El Blog de Tinchicus. https://tinchicus.com/2019/04/09/android-acceso-a-archivos-recursos/

Naresh i Technologies. (2024, 23 de mayo). *A guide to Integer.parseInt() for seamless String to Integer conversion in Java | NareshIT* [Video]. YouTube. https://www.youtube.com/watch?v=HIrAf5XzjdA

Oracle. (s. f.-a). *BufferedReader (Java SE 17 & JDK 17)*. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/BufferedReader.html

Oracle. (s. f.-b). *The while and do-while statements*. The Java Tutorials. https://docs.oracle.com/javase/tutorial/java/nutsandbolts/while.html

Pseintlab. (2026, 6 de septiembre). *¿Qué es una prueba de escritorio?* PSeIntLab. https://pseintlab.com/que-es-una-prueba-de-escritorio/

Shakmuria. (2018, 12 de junio). *176. Curso Java desde cero. Lectura de ficheros (Clase FileReader y BufferedReader)* [Video]. YouTube. https://www.youtube.com/watch?v=YkRB1c8ucGs

W3Api. (2025, 9 de febrero). *Integer.parseInt()*. https://www.w3api.com/Java/Integer/parseInt/